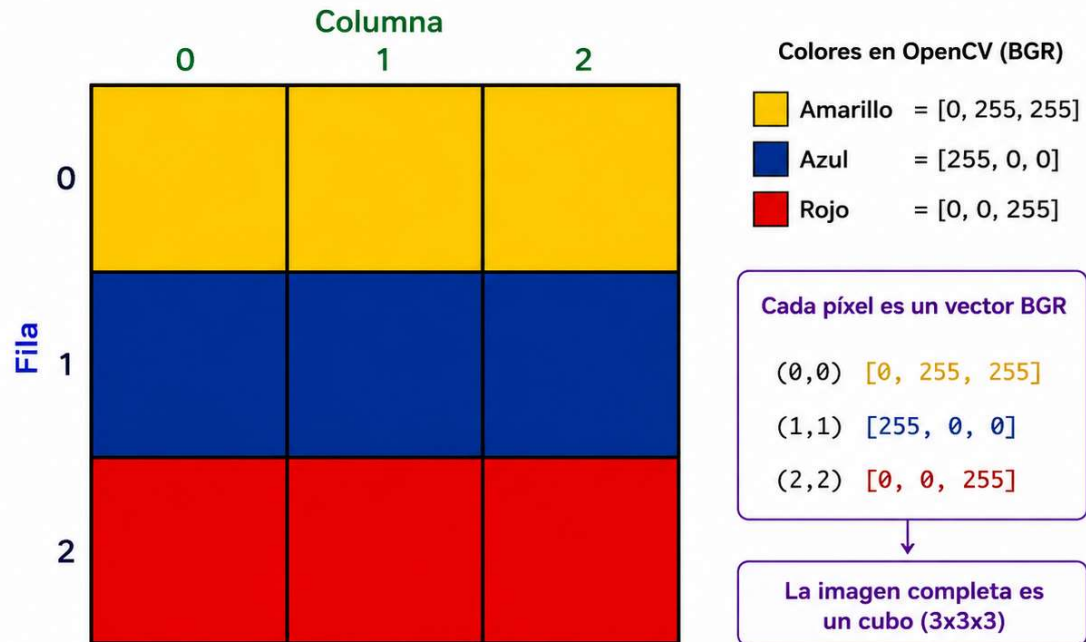


Fundamentos Visión Artificial

felipe.buitrago@ucaldas.edu.co

Imagen de Colombia como un cubo 3x3

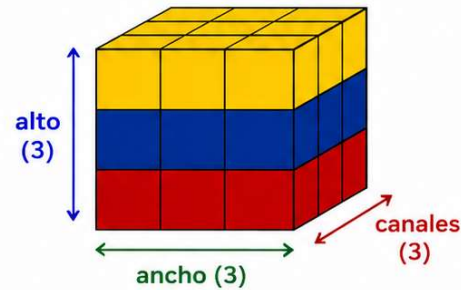
(alto=3, ancho=3, canales=3 BGR)



Forma del cubo (imagen):

(3, 3, 3)

alto
ancho
canales (BGR)



Representación BGR en OpenCV – Bandera de Colombia

Bandera de Colombia (vista normal)



En OpenCV, cada píxel se almacena en orden **BGR** (Azul, Verde, Rojo)

Estructura de un píxel en OpenCV

B	G	R
0 – 255	0 – 255	0 – 255

Valores BGR por cada color de la bandera

Color (real)	Apariencia	Valor en OpenCV (B, G, R)
Amarillo		(0, 255, 255)
Azul		(255, 0, 0)
Rojo		(0, 0, 255)

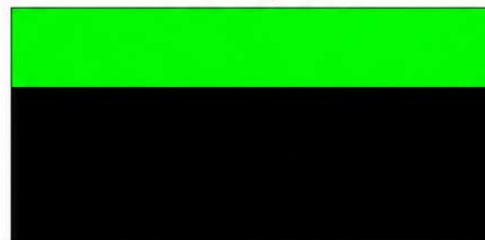
Descomposición de canales en OpenCV (BGR)

Canal B (Azul)



Valores por fila (B):
Amarillo → 0 | Azul → 255 | Rojo → 0

Canal G (Verde)



Valores por fila (G):
Amarillo → 255 | Azul → 0 | Rojo → 0

Canal R (Rojo)



Valores por fila (R):
Amarillo → 255 | Azul → 0 | Rojo → 255

Ejemplo de matriz BGR en OpenCV (3x3 píxeles)

[0, 255, 255]	[0, 255, 255]	[255, 0, 0]	[255, 0, 0]	[0, 0, 255]
[0, 255, 255]	[0, 255, 255]	[255, 0, 0]	[255, 0, 0]	[0, 0, 255]
[0, 255, 255]	[0, 255, 255]	[255, 0, 0]	[255, 0, 0]	[0, 0, 255]
[0, 255, 255]	[0, 255, 255]	[255, 0, 0]	[255, 0, 0]	[0, 0, 255]

Claves
B = Azul
G = Verde
R = Rojo

CONVERSIÓN DE LA BANDERA DE COLOMBIA: DE RGB A HSV

1. BANDERA DE COLOMBIA EN RGB (Representación visual)



RGB: cada píxel tiene 3 canales (R, G, B)

2. BANDERA DE COLOMBIA EN HSV (Representación visual)



HSV: cada píxel tiene 3 canales (H, S, V)

Conversión
RGB → HSV

3. VALORES HSV APROXIMADOS DE CADA COLOR

Color (Real)	RGB (0-255)	HSV (OpenCV)		
		H (0-179)	S (0-255)	V (0-255)
Amarillo	(255, 255, 0)	(30	255	255
Azul	(0, 0, 255)	(120	255	255
Rojo	(255, 0, 0)	(0	255	255

4. FÓRMULAS DE CONVERSIÓN DE RGB A HSV

Sea $R, G, B \in [0, 1]$ (valores normalizados)

Donde:

$$R = \frac{R_{8bit}}{255}, \quad G = \frac{G_{8bit}}{255}, \quad B = \frac{B_{8bit}}{255}$$

$$C_{max} = \max(R, G, B)$$

$$C_{min} = \min(R, G, B)$$

$$\Delta = C_{max} - C_{min}$$

$$H' = \begin{cases} 0, & \text{si } \Delta = 0 \\ 60^\circ \times \frac{(G-B)}{\Delta} \bmod 6, & \text{si } C_{max} = R \\ 60^\circ \times \left(\frac{(B-R)}{\Delta} + 2 \right), & \text{si } C_{max} = G \\ 60^\circ \times \left(\frac{(R-G)}{\Delta} + 4 \right), & \text{si } C_{max} = B \end{cases}$$

$$S = \begin{cases} 0, & \text{si } C_{max} = 0 \\ \frac{\Delta}{C_{max}}, & \text{si } C_{max} \neq 0 \end{cases}$$

$$V = C_{max}$$

Rangos en OpenCV:

$$H \in [0, 179]$$

$$S \in [0, 255]$$

$$V \in [0, 255]$$

(El matiz H en OpenCV es la mitad del valor en grados:

$$H_{opencv} = \frac{H'}{2})$$

5. EJEMPLO DE CÁLCULO PARA LOS COLORES DE LA BANDERA

AMARILLO RGB = (255, 255, 0)

$$R = 1, \quad G = 1, \quad B = 0$$

$$C_{max} = 1, \quad C_{min} = 0, \quad \Delta = 1$$

$$C_{max} = R = G$$

$$H' = 60^\circ \times \frac{(G-B)}{\Delta} \bmod 6 = 60^\circ \times 1 = 60^\circ$$

$$S = \frac{C_{max} - C_{min}}{C_{max}} = \frac{1}{1} = 1$$

$$V = C_{max} = 1$$

$$\text{HSV (grados)} = (60^\circ, 1, 1)$$

$$\text{HSV en OpenCV} = (30, 255, 255)$$

AZUL RGB = (0, 0, 255)

$$R = 0, \quad G = 0, \quad B = 1$$

$$C_{max} = 1, \quad C_{min} = 0, \quad \Delta = 1$$

$$C_{max} = B$$

$$H' = 60^\circ \times \left(\frac{(R-G)}{\Delta} + 4 \right) = 60^\circ \times (0 + 4) = 240^\circ$$

$$S = \frac{C_{max} - C_{min}}{C_{max}} = \frac{1}{1} = 1$$

$$V = C_{max} = 1$$

$$\text{HSV (grados)} = (240^\circ, 1, 1)$$

$$\text{HSV en OpenCV} = (120, 255, 255)$$

ROJO RGB = (255, 0, 0)

$$R = 1, \quad G = 0, \quad B = 0$$

$$C_{max} = 1, \quad C_{min} = 0, \quad \Delta = 1$$

$$C_{max} = R$$

$$H' = 0^\circ$$

$$S = \frac{C_{max} - C_{min}}{C_{max}} = \frac{1}{1} = 1$$

$$V = C_{max} = 1$$

$$\text{HSV (grados)} = (0^\circ, 1, 1)$$

$$\text{HSV en OpenCV} = (0, 255, 255)$$

6. RESUMEN VISUAL

RGB

(Como se ve)



HSV

(Representación HSV)



En HSV, el color se representa por:

- H = tipo de color (matiz)
- S = intensidad del color
- V = brillo del color

Transformación de BGR a HSV en OpenCV

OpenCV lee las imágenes en formato BGR. Para trabajar con color de forma más intuitiva, podemos convertirlas al espacio HSV (Tono, Saturación, Valor).

1. Imagen Original (BGR)



2. Imagen en HSV (compuesta)



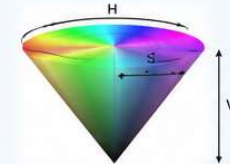
`cv2.cvtColor(imagen, cv2.COLOR_BGR2HSV)`

¿Qué representa HSV?

H (Hue / Tono): color puro (0° a 179° en OpenCV)

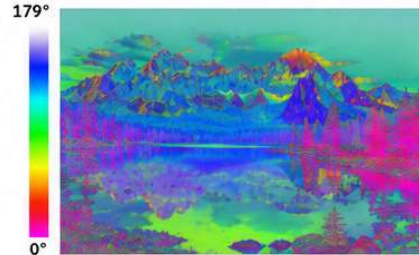
S (Saturation / Saturación): intensidad del color (0 = gris, 255 = color puro)

V (Value / Valor): brillo (0 = negro, 255 = máximo brillo)



3. Canales HSV por separado

H - Tono (Hue)



Representa el tipo de color (rojos, verdes, azules, etc.)

S - Saturación (Saturation)



Indica qué tan "puro" es el color. Blanco = color puro, Negro = gris

V - Valor (Value)



Indica el brillo de la imagen. Blanco = muy brillante, Negro = oscuro

¿Por qué usar HSV?

- ✓ Permite segmentar colores de forma más robusta.
- ✓ Es menos sensible a cambios de iluminación (valor).
- ✓ Ideal para aplicaciones como detección de objetos por color, seguimiento, visión robótica, etc.

Código en OpenCV (Python)

```
import cv2
imagen_bgr = cv2.imread('paisaje.jpg') # OpenCV carga en BGR
imagen_hsv = cv2.cvtColor(imagen_bgr, cv2.COLOR_BGR2HSV) # Conversión a HSV
```

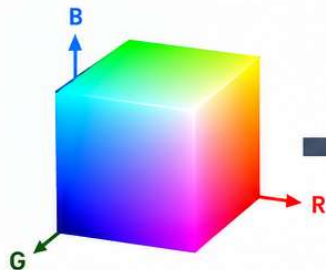
Rangos en OpenCV (8 bits por canal)

- H: 0 - 179 (equivale a 0° - 360°)
- S: 0 - 255 (0 = gris, 255 = color puro)
- V: 0 - 255 (0 = negro, 255 = muy brillante)

CONVERSIÓN DE RGB A CIELAB

El espacio CIELAB ($L^*a^*b^*$) está diseñado para que la distancia entre colores sea perceptualmente uniforme para el ojo humano.

1. RGB (Entrada)



Valores:
 $R, G, B \in [0, 255]$

2. De RGB a XYZ

1. Normalizar RGB a $[0,1]$:

$$r = \frac{R}{255}, \quad g = \frac{G}{255}, \quad b = \frac{B}{255}$$

2. Corrección gamma:

$$C_{lin} = \begin{cases} \frac{C}{12.92} & \text{si } C \leq 0.04045 \\ \left(\frac{C + 0.055}{1.055} \right)^{2.4} & \text{si } C > 0.04045 \end{cases}$$

donde $C \in \{r, g, b\}$

3. Conversión a XYZ (Iluminante D65):

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.4124 & 0.3576 & 0.1805 \\ 0.2126 & 0.7152 & 0.0722 \\ 0.0193 & 0.1192 & 0.9505 \end{bmatrix} \begin{bmatrix} r_{lin} \\ g_{lin} \\ b_{lin} \end{bmatrix}$$

Resultado: X, Y, Z (Tristímulos del color)

3. De XYZ a CIELAB

1. Normalizar por el blanco de referencia (D65):

$$x = \frac{X}{X_n}, \quad y = \frac{Y}{Y_n}, \quad z = \frac{Z}{Z_n}$$

$$(X_n = 95.047, \quad Y_n = 100.000, \quad Z_n = 108.883)$$

2. Función $f(t)$:

$$f(t) = \begin{cases} t^{1/3} & \text{si } t > \left(\frac{6}{29}\right)^3 \\ \frac{1}{3} \left(\frac{29}{6}\right)^2 t + \frac{4}{29} & \text{si } t \leq \left(\frac{6}{29}\right)^3 \end{cases}$$

3. Cálculo de $L^*a^*b^*$:

$$L^* = 116 f(y) - 16$$

$$a^* = 500 [f(x) - f(y)]$$

$$b^* = 200 [f(y) - f(z)]$$

Rangos típicos:

$$L^* \in [0, 100]$$

$$a^* \in [-128, 127]$$

$$b^* \in [-128, 127]$$

Ejemplo de Imagen en RGB



Ejemplo de Imagen en CIELAB (Puro)



¿Qué representa cada componente en CIELAB?

• L^* (Luminosidad)

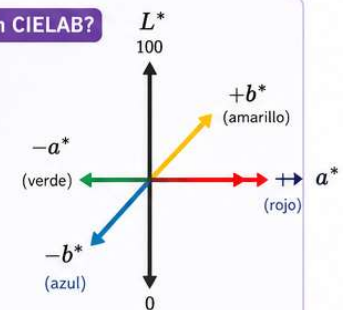
0 = negro, 100 = blanco

• a^* (Componente verde ↔ rojo)

negativo: verde | positivo: rojo

• b^* (Componente azul ↔ amarillo)

negativo: azul | positivo: amarillo



Resumen del flujo: **RGB** → (Normalización + Gamma) → **XYZ** → (Normalización) → **CIELAB ($L^*a^*b^*$)**

OPERACIÓN AND ENTRE DOS IMÁGENES

La operación AND se aplica píxel por píxel.

Imagen 1
(Paisaje)



IMAGEN 1
(Paisaje)

AND

Imagen 2
(Máscara de círculos)

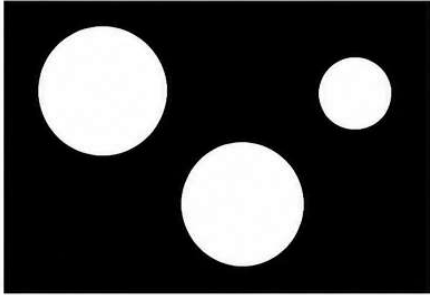


IMAGEN 2
(Máscara de círculos)

=

Resultado
(AND)



RESULTADO (AND)

Ejemplo de píxeles

R	G	B
120	200	150

Valores reales del paisaje
(0 - 255)

Ejemplo de píxeles (máscara)

R	G	B	
255	255	255	– Dentro del círculo (blanco = 255)
R	G	B	
0	0	0	– Fuera del círculo (negro = 0)

Ejemplo de resultado

Dentro del círculo: Se conserva el píxel	R	G	B
	120	200	150
Fuera del círculo: Se elimina el píxel	R	G	B
	0	0	0

¿Cómo funciona?

AND es una operación lógica:

- 1 AND 1 = 1 (se conserva)
- 0 AND 1 = 0 (se elimina)
- 1 AND 0 = 0 (se elimina)
- 0 AND 0 = 0 (se elimina)

En imágenes:

- Blanco (255) actúa como 1 (conserva el píxel)
- Negro (0) actúa como 0 (elimina el píxel)

Fórmula:

$$I_{\text{resultado}}(x, y) = I_1(x, y) \text{ AND } I_2(x, y)$$

Aplicada a cada canal (R, G y B)

Aplicaciones:

- Aplicar máscaras
- Extraer regiones de interés
- Segmentación
- Visión por computadora

¿CÓMO FUNCIONA EN OPENCV?

Para encontrar las diferencias entre dos imágenes, se usa la RESTA ABSOLUTA (absdiff) y luego se aplica un umbral para resaltar los cambios.



Código en OpenCV (Python)

```
import cv2
import numpy as np

# 1. Cargar imágenes
img1 = cv2.imread('imagen1.png') # Imagen original
img2 = cv2.imread('imagen2.png') # Imagen con diferencias

# 2. Asegurar mismo tamaño
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

# 3. Resta absoluta
resta = cv2.absdiff(img1, img2)

# 4. Convertir a escala de grises
gris = cv2.cvtColor(resta, cv2.COLOR_BGR2GRAY)

# 5. Aplicar umbral para resaltar diferencias
_, umbral = cv2.threshold(gris, 30, 255, cv2.THRESH_BINARY)

# 6. Mostrar resultados
cv2.imshow('Imagen 1', img1)
cv2.imshow('Imagen 2', img2)
cv2.imshow('Diferencias', umbral)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

¿Qué hace cada paso?

- 1 Carga la imagen original.
- 2 Carga la segunda imagen con cambios.
- 3 Calcula la diferencia absoluta por pixel:
 $|I_1(x, y) - I_2(x, y)|$
- 4 Convierte la imagen a escala de grises.
- 5 Aplica un umbral para obtener solo las diferencias (blanco = cambio).
- 6 Muestra los resultados.

Explicación

- La función `cv2.absdiff()` calcula la diferencia absoluta entre dos imágenes.
- Los valores cercanos a 0 (negro) son zonas iguales.
- Los valores altos (blanco) son zonas diferentes.
- El umbral convierte esas diferencias en blanco puro para que sean fáciles de ver.

Tabla de ejemplo (un píxel)

Imagen 1 (I1)	Imagen 2 (I2)	Resta abs $ I1 - I2 $	Umbral (>30)
120	120	0	0 (negro)
200	50	150	255 (blanco)
80	90	10	0 (negro)

■ 0 = sin diferencia (negro)
□ 255 = diferencia (blanco)



Conclusión

OpenCV permite detectar diferencias entre dos imágenes usando resta absoluta y umbralización. Es la base de muchos juegos "Encuentra las diferencias" y aplicaciones de visión por computadora.

Aplicaciones

- Juegos infantiles
- Detección de cambios
- Seguridad y vigilancia
- Seguimiento de objetos

Filtros

1. ¿Qué es un filtro en visión artificial?

Un **filtro** es una operación que se aplica a una imagen para modificar sus píxeles con el objetivo de resaltar información importante o reducir ruido. En visión artificial, los filtros se usan para mejorar la calidad de la imagen, extraer bordes, detectar patrones, suavizar, realzar detalles, entre otros.

La mayoría de los filtros trabajan mediante **convolución**, es decir, moviendo una pequeña matriz (**kernel**) sobre cada píxel de la imagen y calculando un nuevo valor a partir de los píxeles vecinos.

2. ¿Qué es el kernel?

El **kernel** (o máscara) es una pequeña matriz de números que define cómo se combinarán los píxeles vecinos.

- Se desplaza por toda la imagen.
- En cada posición, se multiplica elemento a elemento con la vecindad de la imagen.
- Luego se suman los resultados para obtener el nuevo valor del píxel central.

Ejemplo:

Un kernel de 3×3 tiene 9 valores que determinan el comportamiento del filtro.

Vecindad de la imagen (3×3) Kernel (3×3) Resultado

105	110	108
100	102	98
95	90	92

*

1	1	1
1	1	1
1	1	1

=

900

Suma total = 900 → nuevo valor del píxel central

3. Tres filtros importantes en visión artificial

1) Filtro Gaussiano (Suavizado)

Reduce el ruido de la imagen y suaviza los cambios bruscos de intensidad.

Kernel 3×3:

$$\frac{1}{16} \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Ejemplo:

Imagen original



Con filtro Gaussiano



2) Filtro de Sobel (Detección de bordes)

Detecta bordes resaltando los cambios de intensidad en horizontal y vertical.

Kernels 3×3:

Sobel en X (G_x)

-1	0	1
-2	0	2
-1	0	1

Sobel en Y (G_y)

-1	-2	-1
0	0	0
1	2	1

Ejemplo (bordes en X + Y):

Imagen original



Con filtro Sobel



3) Filtro Laplaciano (Realce de bordes)

Resalta los bordes y detalles finos. Muy útil para identificar contornos.

Kernel 3×3:

0	1	0
1	-4	1
0	1	0

Ejemplo:

Imagen original



Con filtro Laplaciano



Filtro Gaussiano



Felipe Buitrago Carmona – Facultad Inteligencia Artificial e Ingenierías – Universidad de Caldas

De imágenes ruidosas
a información útil

Filtro gaussiano

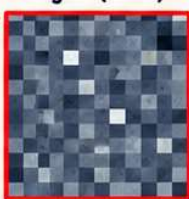
Visión artificial
Imágenes más claras
para mejores decisiones



1. Una imagen puede contener ruido



Ampliación de una
región (ruido)



10	10	10
10	200	10
10	10	10

El ruido produce pequeñas variaciones de intensidad que pueden dificultar el análisis de la imagen.

El valor **200** representa una variación muy fuerte respecto a sus vecinos.

2. El filtro combina cada píxel con sus vecinos

Máscara gaussiana

1	2	1
2	4	2
1	2	1

Todo dividido entre 16

Centro: mayor peso

Vecinos: peso intermedio

Esquinas: menor peso

Cálculo del nuevo valor

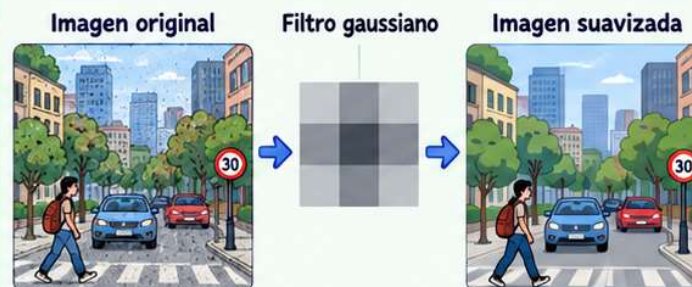
$$\begin{aligned} \text{Nuevo valor} = & (1 \cdot 10 + 2 \cdot 10 + 1 \cdot 10 + \\ & 2 \cdot 10 + 4 \cdot 200 + 2 \cdot 10 + \\ & 1 \cdot 10 + 2 \cdot 10 + 1 \cdot 10) / 16 \end{aligned}$$

$$\text{Nuevo valor} = 920 / 16$$

$$\text{Nuevo valor} = 57.5$$

El valor extremo se reduce al mezclarse con la información de los píxeles vecinos.

3. La imagen queda más suave y estable



El suavizado facilita tareas posteriores:



Facilita:

- Detección de bordes
- Reconocimiento de objetos
- Segmentación
- Análisis de formas

El filtro no busca detectar objetos directamente; prepara la imagen para que otros algoritmos trabajen mejor.

Ruido

Promedio ponderado

Imagen suavizada

Conclusión



El filtro gaussiano reduce variaciones pequeñas de intensidad al reemplazar cada píxel por un promedio ponderado de sus vecinos. Esto produce una imagen más suave y adecuada para etapas posteriores de visión artificial.

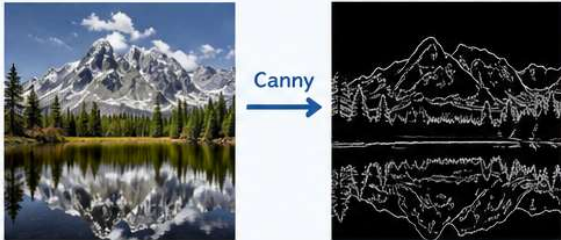
Algoritmo Canny

ALGORITMO DE CANNY

Detección óptima de bordes basada en derivadas

¿QUÉ ES?

El algoritmo de Canny es un método de detección de bordes que identifica cambios bruscos en la intensidad de los píxeles. Utiliza derivadas para encontrar dónde la intensidad cambia significativamente, correspondiente a los bordes de los objetos.



RELACIÓN CON LAS DERIVADAS

La intensidad de la imagen $I(x, y)$ se considera una función. Las derivadas miden cuánto cambia esa intensidad.

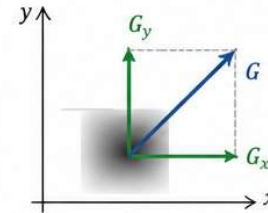
Se calculan las derivadas parciales:

$$G_x = \frac{\partial I}{\partial x}$$

Cambios en horizontal

$$G_y = \frac{\partial I}{\partial y}$$

Cambios en vertical



Luego se calcula la magnitud del gradiente:

$$G = \sqrt{G_x^2 + G_y^2}$$

Indica la fuerza del borde en cada píxel

La dirección del gradiente es:

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

APROXIMACIÓN DE DERIVADAS: OPERADOR SOBEL

Las derivadas se aproximan con convoluciones.

Ejemplo: filtro Sobel horizontal (G_x):

-1	0	1
-2	0	2
-1	0	1

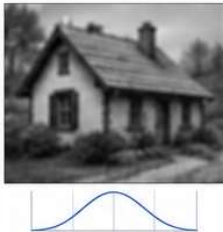
El filtro Sobel vertical (G_y) es la transpuesta del anterior.

-1	-2	-1
0	0	0
1	2	1

ETAPAS DEL ALGORITMO DE CANNY

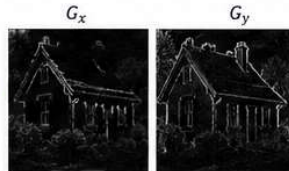
1. Suavizado Gaussiano

Reduce el ruido de la imagen.



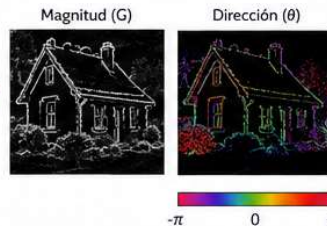
2. Cálculo de derivadas

Se obtienen G_x y G_y usando filtros como Sobel.



3. Magnitud y dirección

Se calcula la magnitud G y la dirección θ del gradiente.



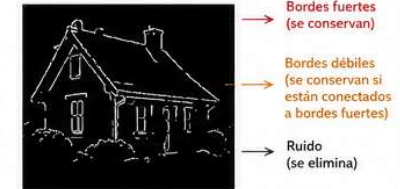
4. Supresión de no máximos

Se conservan solo los máximos locales en la dirección del gradiente.



5. Umbralización por histéresis

Se usan dos umbrales (alto y bajo) para eliminar bordes débiles y ruido.



OBJETIVO DE CANNY

Detectar bordes verdaderos con buena localización, buena detección y mínima respuesta a ruido.



¿POR QUÉ ES EFECTIVO?

Buena detección

Encuentra la mayoría de los bordes reales.



Buena localización

Los bordes detectados están cerca de la posición real.



Mínima respuesta a ruido

Reduce falsas detecciones causadas por ruido.



RESUMEN

Canny utiliza derivadas (gradientes) para encontrar cambios importantes de intensidad y así detectar bordes de manera precisa y robusta.



Algoritmo de Canny: matemáticas paso a paso

Ejemplo con una imagen 3x3 para entender cómo se detecta un borde



Detección de bordes

1. La imagen y el cambio de intensidad

10	10	200
10	10	200
10	10	200

A la izquierda hay valores oscuros y a la derecha valores claros.

El cambio fuerte entre 10 y 200 sugiere la presencia de un borde vertical.

Un borde aparece donde la intensidad cambia bruscamente

2. Cálculo del gradiente con Sobel

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Cálculo de G_x :

$$\begin{aligned} G_x &= (-1 \cdot 10) + (0 \cdot 10) + (1 \cdot 200) \\ &\quad + (-2 \cdot 10) + (0 \cdot 10) + (2 \cdot 200) \\ &\quad + (-1 \cdot 10) + (0 \cdot 10) + (1 \cdot 200) \\ G_x &= -10 + 0 + 200 - 20 + 0 + 400 \\ &\quad - 10 + 0 + 200 \end{aligned}$$

$$G_x = 760$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Cálculo de G_y :

$$\begin{aligned} G_y &= (-1 \cdot 10) + (-2 \cdot 10) + (-1 \cdot 200) \\ &\quad + (0 \cdot 10) + (0 \cdot 10) + (0 \cdot 200) \\ &\quad + (1 \cdot 10) + (2 \cdot 10) + (1 \cdot 200) \\ G_y &= -10 - 20 - 200 + 0 + 0 + 0 \\ &\quad + 10 + 20 + 200 \end{aligned}$$

$$G_y = 0$$

G_x mide el cambio horizontal de intensidad. G_y mide el cambio vertical. Aquí el cambio horizontal es muy grande y el vertical es nulo.

Magnitud del gradiente:

$$\begin{aligned} G &= \sqrt{G_x^2 + G_y^2} \\ G &= \sqrt{760^2 + 0^2} \end{aligned}$$

$$G = 760$$

Dirección del gradiente:

$$\begin{aligned} \theta &= \arctan(G_y / G_x) \\ \theta &= \arctan(0 / 760) \end{aligned}$$

$$\theta = 0^\circ$$

Como G_x es grande y G_y es 0, el algoritmo detecta un borde vertical

3. Decisión: ¿es borde o no?

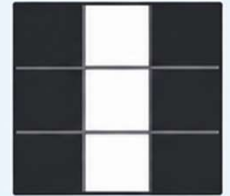
Valor de G	Resultado
Si $G > 300$	borde fuerte
Si $100 < G \leq 300$	borde débil
Si $G \leq 100$	no es borde

Como $G = 760$, este punto se clasifica como borde fuerte

Salida binaria (valores):

0	255	0
0	255	0
0	255	0

Representación visual:



El algoritmo conserva la ubicación donde el cambio de intensidad es más fuerte y marca ese lugar como borde.

Flujo del algoritmo de Canny:

Matriz de entrada



Sobel en x e y



Magnitud y dirección



Umbral



Borde detectado

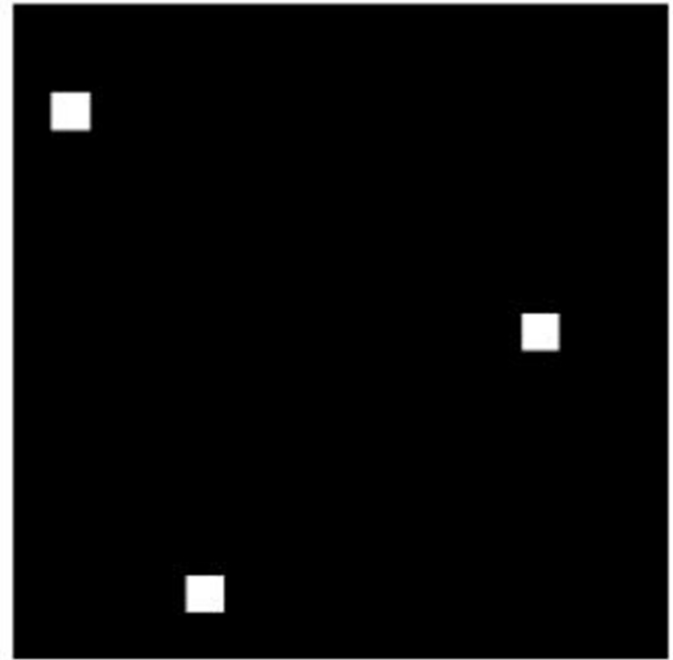
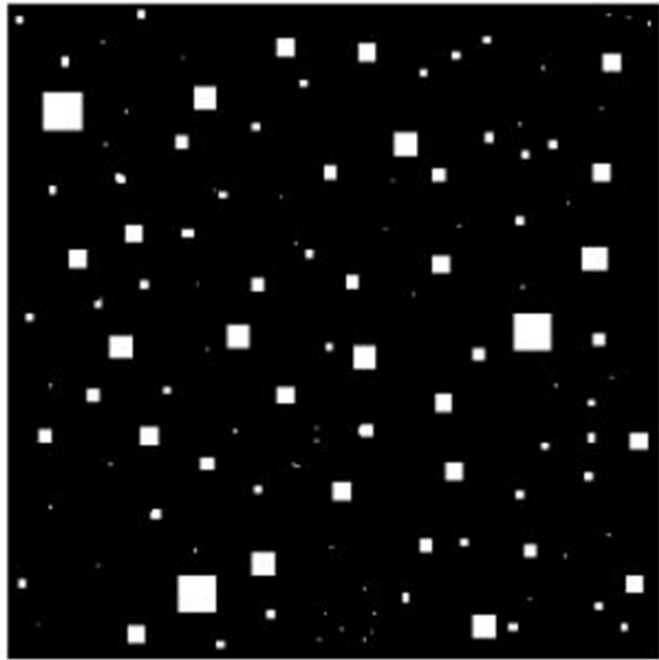


Conclusión:

El algoritmo de Canny identifica bordes al medir matemáticamente los cambios de intensidad entre píxeles. En este ejemplo, el salto de 10 a 200 produce un gradiente alto, por lo que se detecta un borde vertical.

Felipe Buitrago Carmona – Facultad Inteligencia Artificial e Ingenierías – Universidad de Caldas

Operaciones Morfológicas



¿QUÉ ES?

La dilatación agrega píxeles a los bordes de los objetos (valor 1) utilizando un kernel.

DILATACIÓN MORFOLÓGICA

Ejemplo paso a paso con una imagen de 9x9

KERNEL UTILIZADO (3x3)

1	1	1
1	1	1
1	1	1

El kernel define la forma y dirección de la expansión.

CASO 1: CUANDO TODO ES 0

1 Posición inicial del kernel (arriba a la izquierda)

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Como todos los píxeles son 0, el píxel central es 0.

2 El kernel se mueve a la derecha

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Como todos los píxeles son 0, el píxel central es 0.

3 El kernel se mueve hacia abajo y continúa recorriendo la imagen

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Como todos los píxeles son 0, el píxel central es 0.

4 Resultado final: el kernel recorrió toda la imagen y no encontró 1

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

No se agregan píxeles. La imagen permanece igual.

CASO 2: CUANDO EL KERNEL ENCUENTRA UN 1

1 Posición inicial del kernel (arriba a la izquierda)

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

El píxel central es 0. No se agrega ningún píxel.

2 El kernel se mueve y se acerca al 1

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

El píxel central es 0. No se agrega ningún píxel.

3 El kernel se coloca sobre el 1 (el 1 queda en el centro)

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

El píxel central es 1. Se agregan píxeles (dilatación).

4 Resultado después de la dilatación

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

El objeto se expande en todas las direcciones según el kernel.

0

Píxel con valor 0 (fondo)

1

Píxel con valor 1 (objeto)

Posición del kernel (3x3)

Píxel central del kernel

Píxeles agregados por la dilatación

¿QUÉ ES?

La erosión elimina píxeles en los bordes de los objetos (valor 1) utilizando un kernel.

Solo el píxel central se mantiene en 1 si TODOS los píxeles bajo el kernel son 1.

EROSIÓN MORFOLÓGICA

Ejemplo paso a paso con una imagen de 9x9

KERNEL UTILIZADO (3x3)

1	1	1
1	1	1
1	1	1

El kernel define la forma y dirección de la erosión.

CASO 1: CUANDO TODO ES 0

1 Posición inicial del kernel (arriba a la izquierda)

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Como todos los píxeles son 0, NO se cumple la condición.

2 El kernel se mueve a la derecha

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Como todos los píxeles son 0, NO se cumple la condición.

3 El kernel se mueve hacia abajo y continúa recorriendo la imagen

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Como todos los píxeles son 0, NO se cumple la condición.

4 Resultado final: el kernel recorrió toda la imagen y no encontró 1

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

No se elimina ningún píxel porque no hay píxeles con valor 1.

CASO 2: CUANDO EL KERNEL ENCUENTRA UN 1

1 Posición inicial del kernel (arriba a la izquierda)

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0	0
0	0	1	1	1	1	1	1	0
0	0	0	1	1	1	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0

El píxel central es 0.
No se cumple la condición.
No se cambia nada.

2 El kernel se mueve y se acerca al objeto

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	1	1	1	1	1	1	0
0	0	0	1	1	1	1	0	0
0	0	0	1	1	0	0	1	0
0	0	0	0	1	0	0	1	0
0	0	0	0	0	0	0	0	0

El píxel central es 0 porque no todos los píxeles bajo el kernel son 1.
No se cambia nada.

3 El kernel se coloca sobre el centro del objeto (todos los valores son 1)

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Todos los píxeles bajo el kernel son 1.
El píxel central es 1.
Se mantiene el 1 y se eliminan los demás.

4 Resultado después de la erosión

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Solo el píxel central del objeto se mantiene en 1.
Los bordes se eliminan.

0 Píxel con valor 0 (fondo)

1 Píxel con valor 1 (objeto)

Posición del kernel (3x3)

Píxel central del kernel

Píxeles eliminados por la erosión

Detección Figuras Geométricas

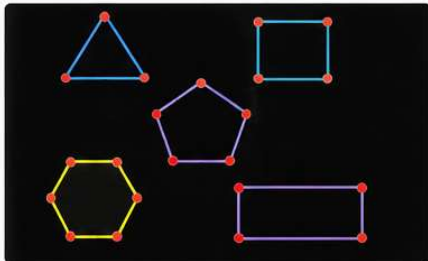
DETECCIÓN DE POLÍGONOS - PASO A PASO (VISIÓN GENERAL)



6 APROXIMACIÓN Y CLASIFICACIÓN DE POLÍGONOS

Cada contorno se aproxima a un polígono y se cuenta la cantidad de lados para clasificar (triángulo, cuadrado, pentágono, etc.).

```
aproximacion = approxPolyDP(contorno)
lados = len(aproximacion)
```



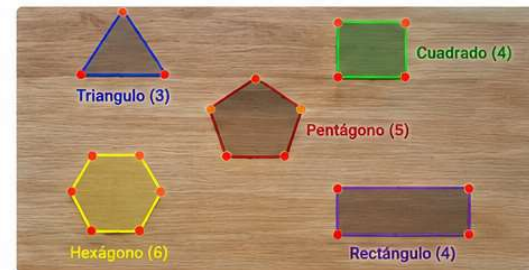
CLASIFICACIÓN POR NÚMERO DE LADOS

- 3** → Triángulo
- 4** → Cuadrado / Rectángulo
- 5** → Pentágono
- 6** → Hexágono
- 7+** → Polígono (n lados)

7 DIBUJAR POLÍGONOS Y RESULTADO FINAL

Se dibujan los polígonos detectados, sus vértices y se muestra el nombre según la cantidad de lados.

```
salida = dibujar_poligonos(frame, contornos, ...)
```



RESUMEN DEL PROCESO

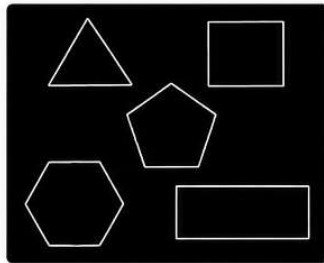
Frame → Gris → Gaussian Blur → Canny (bordes) → Contornos → Aproximación → Clasificación → Dibujo del resultado

APROXIMACIÓN DE CONTORNOS – PROCESO GENERAL

Convierte los contornos detectados en polígonos con menos vértices para poder clasificar formas.

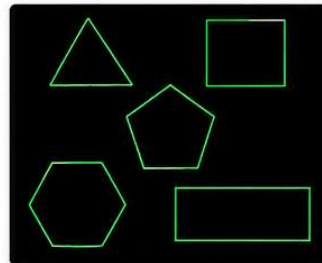
1 IMAGEN BINARIA (BORDES)

Se parte de una imagen binaria donde los bordes de los objetos están definidos (por ejemplo con Canny).



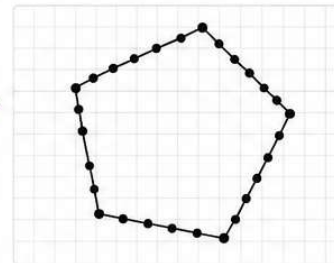
2 ENCONTRAR CONTORNOS

Se detectan los contornos conectados en la imagen usando findContours(). Cada contorno es una lista de puntos.



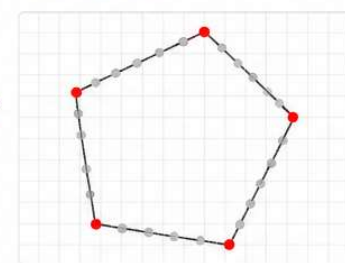
3 CONTORNO COMO CONJUNTO DE PUNTOS

Cada contorno está formado por muchos puntos que siguen el borde del objeto.



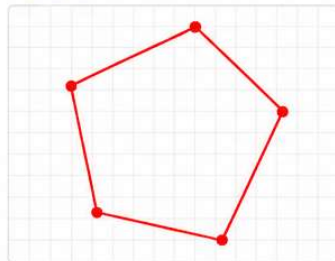
4 SIMPLIFICACIÓN DEL CONTORNO

Se aplica el algoritmo de Douglas–Peucker (implementado en approxPolyDP()), que elimina puntos innecesarios manteniendo la forma general.



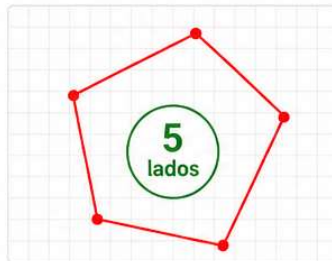
5 CONTORNO APROXIMADO

El resultado es un polígono con menos vértices que representa al objeto original.



6 CONTAR VÉRTICES

Se cuenta la cantidad de vértices del polígono aproximado (número de lados).



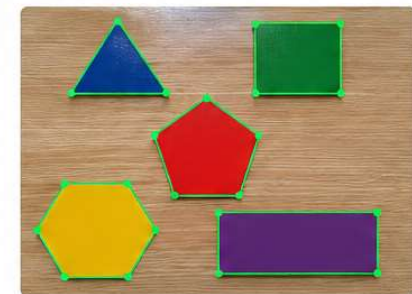
7 CLASIFICAR LA FORMA

Según la cantidad de lados, se clasifica el objeto (triángulo, cuadrado, pentágono, hexágono, etc.).

- 3 → Triángulo
- 4 → Cuadrado / Rectángulo
- 5 → Pentágono
- 6 → Hexágono
- 7 → Heptágono
- 8 → Octágono
- n → Polígono de n lados

8 USO EN LA DETECCIÓN

Se dibuja el polígono aproximado sobre la imagen original para mostrar el resultado.



RESUMEN DEL PROCESO

Bordes (Canny) → findContours() → Contornos (puntos) → approxPolyDP() → Polígono aproximado → Contar lados → Clasificar forma

¿CÓMO FUNCIONA approxPolyDP()?

`approxPolyDP(contorno, epsilon, cerrado)` simplifica un contorno reduciendo la cantidad de vértices mientras mantiene la forma general del objeto.

¿QUÉ ES?

Es el algoritmo de Douglas-Peucker adaptado para polígonos. Elimina puntos intermedios que están "cerca" de la línea que une dos vértices importantes.

PARÁMETRO CLAVE: epsilon

Es la distancia máxima permitida entre un punto del contorno original y el segmento que lo reemplaza.
A mayor epsilon → más simplificación (menos vértices).
a menor epsilon → menos simplificación (más vértices).

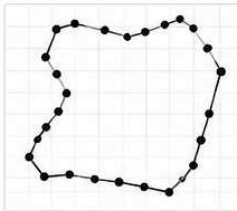
IDEA PRINCIPAL

Conservar los puntos "importantes" (esquinas) y eliminar los puntos que no cambian la forma del contorno más allá de una tolerancia (epsilon).

PASO A PASO DEL ALGORITMO (Douglas-Peucker)

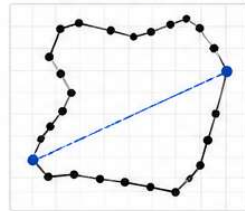
1 CONTORNO ORIGINAL

Tenemos un contorno con muchos puntos.



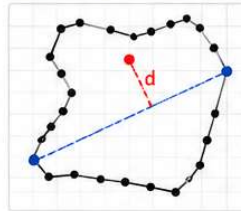
2 UNIR EXTREMOS

Se toma el primer y último punto del contorno y se traza una línea recta entre ellos.



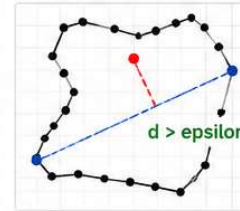
3 DISTANCIA MÁXIMA

Se busca el punto más alejado de esa línea y se mide su distancia perpendicular (d).



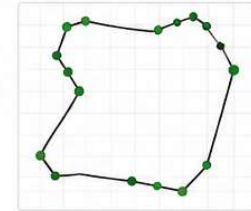
4 COMPARAR CON EPSILON

Si $d > \text{epsilon}$, ese punto se considera importante y se divide el contorno en dos partes.



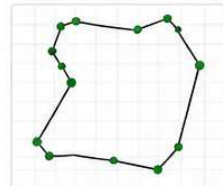
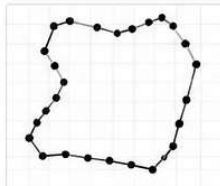
5 REPETIR RECURSIVAMENTE

Se repite el proceso en cada parte hasta que todos los puntos estén a una distancia $\leq \text{epsilon}$.



RESULTADO: APROXIMACIÓN DEL CONTORNO

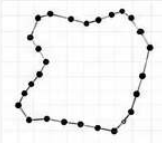
El resultado es un polígono con menos vértices que representa la forma original.



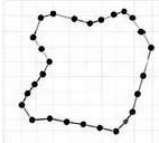
EFEECTO DEL VALOR DE epsilon

A mayor epsilon, más vértices se eliminan.

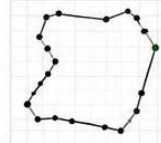
epsilon muy pequeño
(muy detallado)



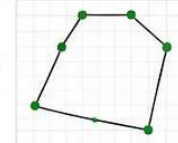
epsilon pequeño



epsilon medio



epsilon grande
(muy simplificado)



EN RESUMEN

`approxPolyDP()` reduce la cantidad de puntos de un contorno manteniendo su forma general, conservando las esquinas y eliminando los puntos innecesarios según la tolerancia epsilon.

Sintaxis:

`approxPolyDP(contorno, epsilon, cerrado)`

- `contorno`: puntos del contorno (lista de puntos)
- `epsilon`: tolerancia (double)
- `cerrado`: True si el contorno está cerrado, False si está abierto

¿QUÉ HACE `cv2.boundingRect(aproximacion)`?

Devuelve el rectángulo alineado con los ejes (horizontal y vertical) más pequeño que contiene completamente al contorno o polígono dado.

SINTAXIS

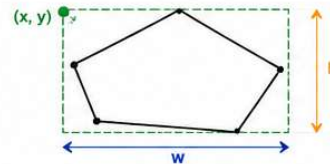
```
x, y, w, h = cv2.boundingRect(aproximacion)
```

Donde devuelve:

- **x**: coordenada X de la esquina superior izquierda
- **y**: coordenada Y de la esquina superior izquierda
- **w**: ancho del rectángulo
- **h**: alto del rectángulo

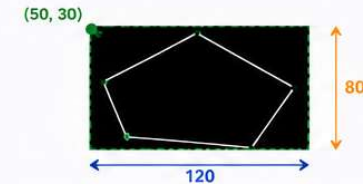
¿QUÉ RETORNA?

Un rectángulo definido por (x, y, w, h) que encierra completamente al contorno.



EJEMPLO

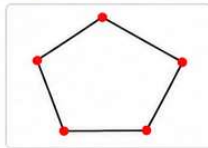
Si `boundingRect` devuelve (50, 30, 120, 80):



¿CÓMO FUNCIONA? (PASO A PASO)

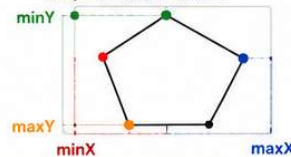
1 CONTORNO APROXIMADO

Tenemos un contorno aproximado (polígono) obtenido con `approxPolyDP`.



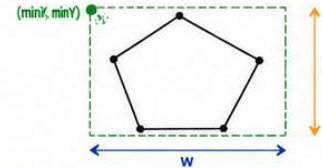
2 BUSCAR EXTREMOS

Se encuentran los valores mínimo y máximo de X e Y entre todos los puntos del contorno.



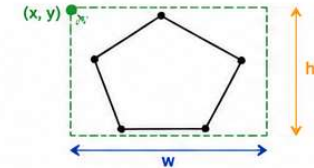
3 DEFINIR RECTÁNGULO

Con esos extremos se forma el rectángulo alineado a los ejes que los contiene.



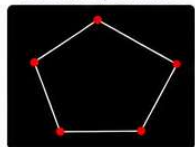
4 DEVOLVER (x, y, w, h)

$x = \text{minX}$, $y = \text{minY}$, $w = \text{maxX} - \text{minX}$, $h = \text{maxY} - \text{minY}$.

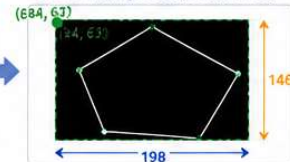


EJEMPLO REAL

Contorno aproximado



`cv2.boundingRect(aproximacion)`



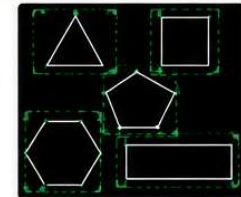
La función devuelve:

(x, y, w, h) = (84, 63, 198, 146)

- **x = 84** → esquina superior izquierda en X
- **y = 63** → esquina superior izquierda en Y
- **w = 198** → ancho del rectángulo
- **h = 146** → alto del rectángulo

¿PARA QUÉ SIRVE?

- ✓ Para obtener la posición y tamaño de un objeto detectado.
- ✓ Para dibujar cuadros (bounding boxes) alrededor de formas.
- ✓ Para recortar (ROI) la región que contiene el objeto.
- ✓ Para calcular relaciones de aspecto, centrar objetos, etc.



EN RESUMEN

`cv2.boundingRect(aproximacion)` calcula el rectángulo alineado a los ejes más pequeño que contiene completamente el contorno o polígono dado y devuelve sus coordenadas (x, y) y dimensiones (w, h).

```
x, y, w, h = cv2.boundingRect(aproximacion)
```